Bayan_CU at IslamicEval 2026: Fine-Grained Hallucination Span Detection in Arabic Quranic and Hadith Texts

Samah Ghonem
Faculty of Computers and Artificial Intelligence, Cairo University, Egypt
samah.a.ghonem@gmail.com

Abstract

Mitigating factual hallucinations in Large Language Model (LLM) responses is critical, especially when handling sacred Islamic text. This paper presents our system for the IslamicEval 2026 Shared Task-Subtask1 (Span Detection), which requires extracting exact character-level spans and fragment labeling of religious citations in Arabic LLM-generated responses. We frame this as a supervised token-classification task, fine-tuning a monolingual bidirectional encoder (AraBERTv2). To process long sequences, we implement a sliding-window chunking mechanism with an overlapping stride. We reconstruct final character boundaries using a post-processing heuristic that resolves overlapping token predictions. Furthermore, we apply a cascading false-positive filter using cross-type conflict resolution and category-specific minimum length thresholds. Our pipeline yields highly competitive results, achieving a Macro-$F1$ score of 0.9652 on the development set and a peak evaluation score of 0.9736 on the official-task test set.

1 Introduction

Despite the promising capabilities of Large Language Models (LLMs) in Arabic Natural Language Processing (NLP), the language's unique syntactic and semantic characteristics pose formidable computational challenges. These bottlenecks stem from high morphological richness, complex prefix/suffix agglutinations, and orthographic ambiguities arising from the typical omission of short-vowel diacritical marks in daily written text (Alayba, 2025). To resolve these complexities, monolingual Pre-trained Language Models (PLMs) such as AraBERT (Antoun et al., 2020) and MARBERT (Abdul-Mageed et al., 2021) leverage bidirectional Transformer Encoder architectures (Devlin et al., 2019) trained on large-scale Arabic corpora. Consequently, these models have achieved state-of-the-art performance on sequence labeling workloads like Named Entity Recognition (NER) (Antoun et al., 2020).

However, generative LLMs still frequently generate factually inconsistent or unfaithful text, known as hallucinations. This phenomenon is highly prevalent in Arabic; evaluations on the AraHalluEval framework indicate significantly higher hallucination rates in Arabic text generation compared to English across multiple reasoning-based LLMs (Alansari and Luqman, 2025).

Mitigating such distortions is paramount when handling sacred religious (Islamic) texts, where precise quotations from canonical sources—the Holy Qur'an and Prophetic Hadiths—are critical. Fabricating, altering, or misattributing the core text (*matn*) or the chain of transmission (*isnad*) severely degrades user trust. To address this, the **IslamicEval 2025** shared task established a dedicated benchmark for detecting and correcting LLM hallucinations in Islamic text (Mubarak et al., 2025).In this paper, we participate in the **IslamicEval 2026** shared task (Alharbi et al., 2026), addressing **Subtask 1 (Span Detection)**. This task requires extracting exact character-level spans of religious quotations within LLM-generated outputs. To solve this, we propose a lightweight, highly competitive sequence-classification and boundary-reconstruction pipeline powered by a fine-tuned AraBERTv2 encoder. [1]

2 Task Definition

Building upon the foundational benchmarks established in the 2025 shared task (Mubarak et al., 2025), the **IslamicEval 2026 Shared Task** organizers (Alharbi et al., 2026) introduce a finer-grained paradigm for evaluating LLM hallucinations. Rather than merely detecting Quranic verses

[1] Our code and post-processing pipeline are publicly available at https://github.com/Samah-G/Bayan_CU-IslamicEval2026.

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1081–1087
October 28-29, 2026 ©2026 Association for Computational Linguistics

and Prophetic Hadith sayings in religious texts, this framework decomposes citations into their core constituent segments.

Specifically, Quranic quotations are segmented into the *Ayah* (the textual verse) and its associated metadata (the *Surah* name and verse number). Hadith citations are similarly parsed into the *matn* (the core prophetic text), the *isnad* (the chain of transmission/reporters), and the primary source or narrator attribution. Isolating these granular, character-level boundary spans allows for a precise inspection of exactly where factual distortions and fabrications occur, facilitating targeted mitigation of hallucinations in sensitive religious contexts.

3 System Architecture and Methodology

We formulate fine-grained hallucination span detection as a supervised token-classification task using a BIO tagging scheme. Rather than using generative decoder-only models, we fine-tune AraBERTv2 (Antoun et al., 2020)—a stacked bidirectional Transformer encoder (Devlin et al., 2019) designed to capture bidirectional context natively. This architecture is crucial for fine-grained Arabic sequence labeling because it directly handles complex prefix/suffix agglutinations and morphology.

Furthermore, while generative pipelines at IslamicEval 2025 relied on complex, error-prone string-alignment algorithms (such as Needleman-Wunsch) to resolve character offsets from generated text (Omayrah et al., 2025), our token-space approach bypasses these alignment bottlenecks entirely. The complete pipeline is illustrated in Figure 1.

3.1 Preprocessing and Sliding-Window Tokenization

To address class imbalance, we filter the training set by discarding responses that do not contain target spans. Because the average LLM-generated response length is 1,460 characters in the training data set, standard encoding risks context truncation. We resolve this by applying a sliding-window tokenization strategy.

We tokenize input texts using the pre-trained AraBERTv2 tokenizer (Antoun et al., 2020) via the Hugging Face `transformers` library (Wolf et al., 2020) to retrieve token IDs and character offset mappings. Sequences are divided into overlapping chunks of maximum-length $L_{\text{max}} = 512$ tokens. To prevent span boundary truncation, we apply a sliding stride (S). We set $S = 128$ tokens for training and evaluate $S \in \{128, 256, 384\}$ during development and testing phases.

3.2 Sequence Labeling and Model Training

For each chunk, gold-truth character offsets are mapped to a 9-class BIO schema:
$\mathcal{Y} = \{\textbf{O}, \textbf{B/I-claimed source}, \textbf{B/I-Ayah}, \textbf{B/I-matn}, \textbf{B/I-isnad}\}$.
We fine-tune the `bert-base-arabertv2` encoder (~110M parameters) by appending a linear classification head over the token hidden representation to predict the token labels $y_t \in \mathcal{Y}$.

Training is executed for 3 epochs with a learning rate of 3×10^{-5} using a batch size of 16. The complete training configuration and parameter rationales are detailed in Table 5 in Appendix A.

3.3 Span Reconstruction Heuristic

During inference, sequences are processed into unlabeled chunks, yielding token-level predicted tags via

$$\hat{y}_t = \arg\max(\text{logits}_t)$$

. We map $\hat{y}_t$ back to character-level boundaries using a three-step heuristic: (1) **Span Merging:** consecutive tokens of category c starting with B-c followed by I-c are merged into a single span spanning from the start of the B token to the end of the last consecutive I token; (2) **Orphaned Tags:** isolated I-c tags lacking a preceding B-c are treated as single-token spans; and (3) **Negative Space:** all O tags are discarded. A visual example of this token-to-offset reconstruction is detailed in Figure 2 in Appendix B.

3.4 Overlapping Span Merging Heuristic

Because input responses are processed in overlapping sliding-window chunks, predictions from adjacent chunks often produce redundant or partially duplicate spans. To reconstruct a single, cohesive character-level span layout, We group all predicted spans by response ID and sort them chronologically by their starting offset (s). We then iteratively compare consecutive spans of identical segment types—defined as span_1 with boundaries $[s_1, e_1]$ and span_2 with boundaries $[s_2, e_2]$—under a dual-conditional merging framework. Spans are merged if they satisfy at least one of the following:

1) Intersection over Union (IoU) Threshold: The overlap represents a significant fraction of their

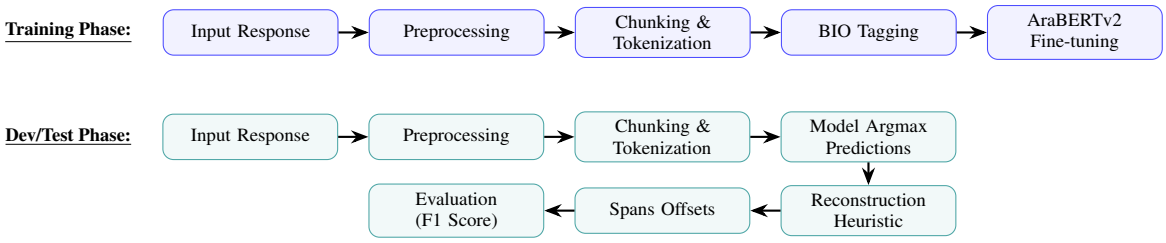


Figure 1: Split-phase system architecture layout showing offline model fine-tuning (top) and online post-processed inference evaluation (bottom).

total combined coverage, defined as:

$$\text{IoU}(span_1, span_2) = \frac{\max(0, \min(e_1, e_2) - \max(s_1, s_2))}{\max(e_1, e_2) - \min(s_1, s_2)} \geq 0.3 \quad (1)$$

2) Boundary Proximity Threshold: To capture short nested spans that would otherwise fail the denominator-heavy IoU check, we merge spans if their start boundaries satisfy:

$$\Delta_{\text{start}} = |s_1 - s_2| \leq 5 \text{ characters} \quad (2)$$

If either condition is met, the spans are merged into a single span with boundaries $[\min(s_1, s_2), \max(e_1, e_2)]$. This merged span replaces both original instances in the queue and is compared against the next span in the sequence. Responses containing no predicted spans are assigned a default `No Annotation` label. Detailed merging calculations are provided in Appendix C.

3.5 False Positive Filtration

To mitigate spurious or hallucinated boundary predictions, we apply a two-step cascading filter:

1. **Cross-Type Conflict Resolution:** Reconstructed spans for a given response are sorted in descending order by character length. If consecutive spans of conflicting segment types overlap (IoU > 0.1), the longer span is preserved and the shorter is discarded.

2. **Minimum Length Constraint:** To eliminate isolated token noise, we enforce a category-specific minimum character-length threshold (T_{len}) and discard spans falling below it:

$$T_{\text{len}} = \begin{cases} 5 & \text{if type} = \texttt{claimed_source} \\ 10 & \text{if type} \in \{\texttt{isnad}, \texttt{Ayah}\} \\ 15 & \text{if type} = \texttt{matn} \end{cases} \quad (3)$$

3.6 Annotation Grouping

Finally, the remaining filtered spans are grouped into compound annotation units according to the task schema: a Quranic *Ayah* is paired with its corresponding *Claimed Source* under a single annotation ID, while a Hadith *Matn*, *Isnad*, and *Claimed Source* are unified under another unique ID.

3.7 Inference and Test Set Evaluation Pipeline

During evaluation, blind test responses pass through the identical processing chain: (1) **Preprocessing & Sliding-Window Tokenization** using identical parameters ($L_{\max} = 512, S \in \{128, 256, 384\}$); (2) **Model Inference** to predict token-level tags via $\hat{y}_t = \arg\max \text{logits}_t$; and (3) **Post-Processing** to apply span reconstruction, overlap merging, and cascaded filtration. The finalized character-level offsets are then evaluated against the gold annotations using the official character-level macro-averaged F1 score.

4 Experimental Results and Analysis

We evaluate our proposed fine-grained hallucination span detection system across several configurations, training hyperparameters, and post-processing setups. The primary evaluation metric is the character-level macro-averaged F_1 score.

4.1 Development Set Results & Hyperparameter Search

We conduct a systematic grid search on the development set, varying model backbones, learning rates (η), fine-tuning epochs (E), and sliding-window strides (S). All results in Table 1 are reported with post-processing filtration applied.

Hyperparameter Insights: (1) **Learning Rate Sensitivity:** $\eta = 3 \times 10^{-5}$ yields optimal convergence. Slower rates underfit (0.9574), whereas higher rates (5×10^{-5}) destabilize updates (0.9622). (2) **Window Stride:** Expanding S from 128 $\rightarrow$ 384 provides vital overlapping context near chunk

Model Backbone	LR (η)	Ep.	Stride (S)	Dev F_1
Impact of Learning Rate (Fixed $E = 3, S = 384$)				
AraBERTv2-base	2×10^{-5}	3	384	0.9574
AraBERTv2-base	$\mathbf{3 \times 10^{-5}}$	**3**	**384**	**0.9652**
AraBERTv2-base	5×10^{-5}	3	384	0.9622
Impact of Window Stride (Fixed $\eta = 3 \times 10^{-5}, E = 3$)				
AraBERTv2-base	3×10^{-5}	3	128	0.9649
AraBERTv2-base	3×10^{-5}	3	256	0.9647
AraBERTv2-base	$\mathbf{3 \times 10^{-5}}$	**3**	**384**	**0.9652**
Impact of Training Epochs (Fixed $\eta = 3 \times 10^{-5}, S = 384$)				
AraBERTv2-base	$\mathbf{3 \times 10^{-5}}$	**3**	**384**	**0.9652**
AraBERTv2-base	3×10^{-5}	4	384	0.9640
Alternative Pre-trained Encoders				
MarBERT	3×10^{-5}	3	384	0.9645
AraBERT-Large	2×10^{-5}	4	128	0.9609
AraBERT-Large	2×10^{-5}	4	384	0.9606

Table 1: Systematic hyperparameter search and model architecture comparison on the development dataset (with full post-processing filtration applied).

edges, boosting F_1 from 0.9649 to 0.9652 while reducing the incidence of broken character spans. (3) **Epoch Limits:** Fine-tuning for 3 epochs outperforms 4 epochs (0.9640), indicating localized overfitting on repetitive syntactic phrases in the chunked dataset. (4) **Model Backbone:** The monolingual AraBERT-base outperforms both the dialect-aware MarBERT (0.9645) and the over-parameterized (340M) AraBERT-Large (0.9609), highlighting the parameter efficiency of base-sized models on MSA sequence labeling. While larger models tend to overfit on the specialized domain.

4.2 Filtration Ablation Study

Before applying post-processing filters, a confusion matrix analysis of our top development models revealed high false-positive rates caused by spurious, isolated token activations. Enforcing our cascading filtration steps (cross-type conflict resolution and minimum-length thresholds) yielded substantial performance recoveries across configurations: A comprehensive confusion matrix and per-class error breakdown on the development set are provided in Appendix D.

Model Configuration	Raw F_1	Post-Filter F_1
AraBERTv2-base ($E = 4, S = 384$)	0.9634	0.9640
AraBERTv2-base ($E = 3, S = 384$)	0.9649	**0.9652**

Table 2: F_1-score gains after applying filtration

4.3 Official Test Set Evaluation

Our final system configurations were evaluated blind on the official Codabench testing platform across three progressive submissions shown in Table 3: Enabling our cascading filtration pipeline on the same base model in Sub.1 boosted the test F_1 score to **0.9706** ($+0.0015$ gain), validating the noise-suppression heuristic. While the best overall test F_1 score **0.9736** resulted from combining the optimal model parameters, with the post-processing filteration steps.

Sub.	Model Backbone & Config	Post-Filter	Test F_1 Score
1	AraBERTv2-base ($E = 4, S = 384$)	Off	0.9691
2	AraBERTv2-base ($E = 4, S = 384$)	On	0.9706
3	**AraBERTv2-base** ($E = 3, S = 384$)	**On**	**0.9736**

Table 3: Official evaluation results of the test-set submissions on the Codabench platform.

4.4 Key Evaluation Trends

Our blind test results reveal three critical performance dynamics: (1) **Generalizability:** Achieving a peak test F_1 score of **0.9736** (surpassing development results) demonstrates strong out-of-domain generalization without data leakage. (2) **Filtration Impact:** Applying cascading post-processing filters on the 4-epoch model yields a direct $+0.0015$ improvement ($0.9691 \rightarrow 0.9706$), proving our heuristics generalize robustly to unseen test instances. (3) **Epoch Optimization:** Mirroring our development findings, reducing fine-tuning epochs from 4 to 3 increases test F_1 by $+0.0030$ to **0.9736**, establishing our 3-epoch, stride-384, post-filtered configuration as our optimal pipeline.

5 Conclusion

We presented an efficient sequence-labeling pipeline using AraBERT for fine-grained religious hallucination span detection in the IslamicEval 2026 shared task. Our token-classification formulation bypasses the alignment bottlenecks and boundary errors of generative decoder-only architectures. Additionally, our post-processing pipeline—incorporating dual-criteria span merging, cross-type conflict resolution, and length filters—substantially mitigates false positive boundaries. Future work will explore dialect-aware language models and ensemble voting across multi-scale sliding windows to improve boundary robustness.

Limitations

Despite the high macro-averaged F_1 performance of our system, several limitations warrant discussion. First, our pipeline is highly optimized for Modern Standard Arabic (MSA), the primary medium of canonical religious texts. Performance may degrade on informal regional dialects or noisy user-generated content where non-standard spellings of source attributes or colloquial phrases are used. Second, the sliding-window chunking strategy introduces structural dependencies; while the 128-token stride successfully mitigates boundary-severing issues, it increases the total inference overhead. Third, our post-processing filter utilizes static, hardcoded minimum length thresholds for segment types (e.g., a threshold of 15 characters for *matn*). While highly effective for filtering out random token-classification noise on this benchmark, these thresholds are heuristic-based and may over-filter authentic, extremely short quotations or source citations in out-of-domain evaluation contexts.

References

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT & MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7088–7105, Online. Association for Computational Linguistics.

Aisha Alansari and Hamzah Luqman. 2025. AraHalluEval: A fine-grained hallucination evaluation framework for Arabic LLMs. In *Proceedings of The Third Arabic Natural Language Processing Conference*, pages 148–161, Suzhou, China. Association for Computational Linguistics.

Abdulaziz M. Alayba. 2025. Arabic natural language processing (nlp): A comprehensive review of challenges, techniques, and emerging trends. *Computers*, 14(11).

Rahaf Alharbi, Abdulelah Alturki, Watheq Mansour, Rana Malhas, Hamdy Mubarak, Kareem Darwish, Tamer Elsayed, and Walid Magdy. 2026. IslamicEval 2026: The second shared task of capturing LLMs hallucination in Islamic content. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 9–15, Marseille, France. European Language Resources Association.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Hamdy Mubarak, Rana Malhas, Watheq Mansour, Abubakr Mohamed, Mahmoud Fawzi, Majd Hawasly, Tamer Elsayed, Kareem Mohamed Darwish, and Walid Magdy. 2025. IslamicEval 2025: The first shared task of capturing LLMs hallucination in islamic content. In *Proceedings of The Third Arabic Natural Language Processing Conference: Shared Tasks*, pages 480–493, Suzhou, China. Association for Computational Linguistics.

Arwa Omayrah, Sakhar Alkhereyf, Ahmed Abdelali, Abdulmohsen Al-Thubaity, Jeril Kuriakose, and Ibrahim AbdulMajeed. 2025. HUMAIN at IslamicEval 2025 shared task 1: A three-stage LLM-based pipeline for detecting and correcting hallucinations in Quran and Hadith. In *Proceedings of The Third Arabic Natural Language Processing Conference: Shared Tasks*, pages 509–514, Suzhou, China. Association for Computational Linguistics.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

A Hyperparameter Configurations

This appendix illustrates the Model training hyperparameter settings and selection rationales in Table 5.

B Token-to-Offset Mapping Example

This appendix illustrates the token-to-offset span reconstruction heuristic, mapping predicted token boundaries to localized Arabic text slices. Figure 2 demonstrates how individual token tags (B-, I-, O) are processed to construct the final character-level offsets for a Quranic *Ayah* span.

Segment Type	Total GT	Correct (%)	Missed / FN (%)	Confused / Wrong (%)	Extra / FP
`Ayah`	698	98.4% (687)	0.9% (6)	0.7% (5)	24
`claimed_source`	1,238	96.0% (1,188)	3.9% (48)	0.2% (2)	68
`isnad`	204	87.3% (178)	9.8% (20)	2.9% (6)	34
`matn`	588	98.3% (578)	1.0% (6)	0.7% (4)	23

Table 4: Per-class error and classification accuracy breakdown relative to ground-truth spans on the development set.

Hyperparameter	Value	Rationale
Max Length (L)	512	Fits long contextual windows
Stride (S)	128	Prevents boundary context truncation
Batch Size	16	Efficient VRAM usage
Epochs	3	Prevents overfitting on chunks
Learning Rate	3×10^{-5}	Optimal for AraBERTv2 fine-tuning

Table 5: Model training hyperparameter settings and selection rationales.

Tokens	قال	تعالى	إن	الله	مع	الصابرين
Tags	`O`	`O`	`B-Ayah`	`I-Ayah`	`I-Ayah`	`I-Ayah`
Offsets	$(0, 3)$	$(4, 9)$	$(10, 12)$	$(13, 17)$	$(18, 20)$	$(21, 29)$
Action	*Discarded*		**Span:** `Ayah` $(10, 29)$			

Figure 2: Token-to-offset span reconstruction mapping.

C Span Merging Numerical Examples

This appendix provides concrete examples of the dual-conditional merging framework applied to consecutive predicted segments of identical types during the post-processing phase:

- **Example 1 (No Merge via IoU):** Given $\text{span}_1 = [5, 30]$ and $\text{span}_2 = [20, 50]$, the intersection is $30 - 20 = 10$, and the union is $50 - 5 = 45$.

$$\text{IoU} = \frac{10}{45} \approx 0.22 < 0.3$$

  Because the start boundary delta is $|5 - 20| = 15 > 5$, neither condition is met, and the spans are preserved as separate.

- **Example 2 (Successful Merge via IoU):** Given $\text{span}_1 = [10, 40]$ and $\text{span}_2 = [15, 55]$, the intersection is $40 - 15 = 25$, and the union is $55 - 10 = 45$.

$$\text{IoU} = \frac{25}{45} \approx 0.55 \geq 0.3$$

  This triggers a successful merge, resulting in a single reconstructed span spanning $[10, 55]$.

- **Example 3 (Successful Merge via Proximity):** Given a long reference $\text{span}_1 = [10, 70]$ and a short nested $\text{span}_2 = [15, 25]$, the intersection is $25 - 15 = 10$, and the union is $70 - 10 = 60$.

$$\text{IoU} = \frac{10}{60} \approx 0.167 < 0.3$$

  Although it fails the IoU threshold due to the long union length, the absolute difference between their start offsets is:

$$\Delta_{\text{start}} = |10 - 15| = 5 \leq 5$$

  This satisfies the proximity threshold, successfully merging the segments into a single span spanning $[10, 70]$.

D Detailed Development Error Analysis

In this appendix, we present the detailed error patterns of our fine-tuned AraBERT sequence labeler (3×10^{-5} learning rate, 3 epochs, 384 stride) evaluated on the development set after applying our final cascaded filters.

D.1 Error Classification and Matrix Analysis

Table 6 categorizes the precise types of boundary and classification errors observed across all predicted spans. The model achieves 2,459 exact span matches, while boundary shifting issues (split across minor shifts of 1–5 characters and major shifts of > 5 characters) account for a total of 172 errors. The largest source of performance degradation is hallucinated spans (149 False Positives compared to 258 False Positives without the dual-filteration steps), justifying our cascaded post-processing filtering.

Table 7 provides the token-level confusion matrix among our four primary segment classes: *Ayah*, *claimed_source*, *isnad*, and *matn*, alongside instances that were entirely missed (False Negatives). Table 4 contextualizes these errors as percentages of the total ground-truth (GT) annotations per segment type.

Metric / Error Category	Count
Exact Span Matches	2,459
Missed Spans (FN)	80
Wrong Entity Type	17
Minor Boundary Shift (1–5 chars)	82
Major Boundary Shift (>5 chars)	90
Hallucinated Spans (FP)	149

Table 6: Detailed development set error classification summary.

	Predicted Category				
Ground Truth	**Ayah**	**claimed_ source**	**isnad**	**matn**	**Missed**
Ayah	687	2	0	3	6
claimed_source	0	1,188	0	2	48
isnad	0	4	178	2	20
matn	2	1	1	578	6

Table 7: Token-level prediction confusion matrix on the development set (Rows: Ground Truth, Columns: Predictions).

D.2 Key Qualitative Observations

Our analysis of the per-class performance reveals several distinct syntactic and structural patterns:

- **High Fidelity on Quranic and Hadith's Matn Spans:** Both *Ayah* and *matn* segments achieve exceptionally high correct-type classification rates of 98.4% and 98.3% respectively. This demonstrates the model's outstanding capability to recognize the highly stylized linguistic structures and traditional opening/closing formulas associated with Quranic verses and canonical Prophetic text.
- **Vulnerability of Isnad Detection:** The *isnad* (chain of transmission) segment presents the lowest correct detection rate (87.3%) and the highest missed rate (9.8%). This is primarily due to the morphological diversity and highly variable length of transmission chains (e.g., lists of sequential narrator names), leading to boundary under-segmentation.
- **Spurious Source Citations:** Out of 149 total hallucinated spans, the *claimed_source* category accounted for the largest fraction (68 instances). The model frequently flagged common reference formatting, Surah names, or general vocabulary as actual quoted sources even when they did not reference a specific span. This empirical finding strongly motivates extra post-processing filters to eliminate isolated, short-length *claimed_source* spans.